

Nikhil Mattu

So You Want to Go From FORTRAN (or Visual Basic) to Python (and notes on using VS CODE + VS Community)

Note: Visual Basic code is very similar to Fortran, so much of what is below is applicable to both. Syntax just varies slightly between VB and Fortran

General differences:

- Naming
 - Fortran often uses caps, but is generally case-insensitive
 - Python is case-sensitive
- Indenting
 - Fortran does not factor in indentation when running; instead operates on keywords (e.g. for DO loops, must have an END DO to indicate end)
 - In Python, indentation matters (e.g. with a for-loop, everything indented right after it is considered part of the loop, and it is closed by having code in line with the initial for-loop definition)
- Indexing (same for loops)
 - Fortran starts at 1 and uses ()
 - e.g. SECOND_ELEMENT=ARR(2)
 - Python starts at 0 and uses []
 - e.g. second_element=arr[1]
 - When translating from fortran, be mindful about values used to index
- Loops (DO/for; DO WHILE / while)
 - Fortran is inclusive of the end bound; you must specify start and end (e.g. 1, END); You also need an END DO to close loops as indenting does not matter functionally – the END DO is very important to be aware of, especially if it is located in under a label block other than the one where DO was started
 - e.g DO I=1, END → loops from i=1 to i=END
 - Something like 'DO I=1,1' will loop once
 - Python is NOT inclusive of end bounds; you do not need to specify a starting bound of 0 if starting from 0
 - e.g. for i in range(END) → loops from i=0 to i=END-1
 - Be mindful of the bounds you set when translating from fortran, especially in cases where the loop variable (e.g. 'i') is used within the loop; this ties in with being mindful about indexing
 - Something like 'for i in range(0,1)' will loop once (0,0 would not execute)

GO TO's (and GoTo in VB) vs. Function Calls

In **Python**, you can use calls to defined functions, whereby, upon completion of a called function (e.g. `otherfcn()`), the code returns back to where the function was called to finish up any remaining code (e.g. end of code).

e.g.

```
def main():  
    (start of code)  
    otherfcn()  
    (end of code)  
  
otherfcn():  
    (code)
```

****For variable values that we want to continue into other blocks of code, we need to set variables as global (easier set-up for all-around access to variable values) OR setup functions to take arguments so you can pass variable values over (ideal for passing information about conditions/states of the code – e.g. ‘this is the second time we have been here’ or ‘this value now equals this’ etc. -- and loop variable values when you need to pick back up at the loop iteration you left at after a function call / GO TO)**

- ➔ With setting globals, you likely have to define variables as global in every function those variables are used – if they are only defined once as global, a given function without the global definition may not know to use the variable’s global value
 - That being said, if the variable value is a single element (e.g. not a data structure like an array, but a number, string, etc.), if it is used (lets call this variable `var1`) to set another variable’s value (lets call this variable `var 2`, so e.g. `var2=7*var1 + 30`) as opposed to setting a value for this variable (`var1`; e.g. `var1=90`), then the global definition is *not* actually needed in the given function → in the second case, `var1=90`, the global definition *is* needed if you want `var1`’s assignment to 90 to be reflected in `var1` elsewhere (for a global version of `var1`)

With GO TO in **Fortran**, on the other hand, these act as new paths to take. Essentially they are function calls (to go to whatever label number it specifies and the code there is the “function called”), but they do not return automatically. In the following example code, if the `IF(INTEG...)` is entered, the code will jump to 250 code and that is it. The else code and more body code will not execute

e.g.

| Program prog

```

|      (start of code)
|      IF(INTEGEREXAMPLE.EQ.0) THEN
|          GO TO 250
|      ELSE
|          (else code)
|      (more body code)
|
250 |      (250 code)

```

Translating Fortran GO TO's to Python:

Main Method: Because function calls are similar to GO TO's, except they return to where they were called, you can use them as replacements, BUT you must have something that terminates code after a given function call. If a function call is the very last line of the current function, no extra steps need to be taken, however, if a function call meant as a GO TO substitute that is NOT the last line of the current function will require "return" statements immediately after the call. This method of using "return" statements will make sure that call stacks are cleared over the course of using the program (i.e. each time the program restarts something)

Alternative Method (easier, but suitability depends on program): Use function calls for every GO TO and terminate program (e.g. `sys.exit()`) upon reaching then end. Problem with this is that a call stack will build up throughout use of your program, which can cause issues for programs that have a reset/restart protocol (for starting the program again from the beginning). If you have a "run once" kind of program, this alternative method is probably better to use versus the return-method

- If/else or if/if(check): Assuming a GO TO is used in an IF statement, you could then have your function call in the corresponding if-statement -- all code following the GO TO you would put in an else
 - If the IF with GO TO is in a loop, in python you would have to set a check variable to see if you entered the IF (meaning you would use the function call) and then could do multiple different things:
 - Could put a break after the function call inside the if (assuming only within one loop – really we want to break out of all loops that the if-statement is in and stop execution of any following code), and then outside the loop put the rest of the code in an if (checkvariable==false; assuming false means you did not use the function call

- Note: Suppose we have a GO TO 500. If this GO TO is not within a conditional AND there are no other places in which there is a GO TO 500, then there is no need to define a new function + call for label 500. This is because in this case, the GO TO is solely being used to connect two pieces of the same block of code. This is effectively the same as just moving the code from label 500 to wherever it was called with GO TO 500. **HOWEVER**, below the GO TO 500, there may be code leading up to label 500. If this code continues onto label 500, this would either be one reason to separate out label 500 as a new function OR to just repeat the code (another reason to make a new function instead is if there is a decent amount of code after 500 to be repeated) for label 500 under whatever label function is right before the call to 500... example:

Fortran:

```
100 ... coding1
      GO TO 500
```

```
200 ... coding2
```

```
500 ... coding3
```

Python option 1:

```
def label100to500():
    ... coding1
    ... coding3
    label200()
```

```
def label200():
    ...coding2
    ...coding3
```

Python option 2:

```
def label100:
    ... coding1
    GO TO 500
    label500()
    label200()
```

```
def label200():
    ...coding2
def label500():
    ...coding3
```

File handling (OPEN, WRITE, READ)

****OPEN, WRITE, and READ** have ERR specifiers with them. These are like GO TO's but for if an error occurs in trying to use any of the three functions.

****Some unit numbers** are considered "standard values" like with standard input (5; this number is assumed when using * in place of a unit number for READ), standard output (6; this number is assumed when using * in place of a unit number for WRITE), and standard error (0; this prints, however, it does so to the standard error stream, NOT the usual standard output stream → can make equivalent print() to the standard error string in python using print('Message', Variable, **file=sys.stderr**) – will require import of 'sys' package

OPEN in fortran →

e.g. `OPEN(10, ERR=530, FILE=filename, IOSTAT=IOS, STATUS='OLD')`

Where 10 is the assigned unit number to the file "filename" and OLD indicates that it is an existing file (NEW would instead create and open a new file named "filename" with unit number 10; just to note, there are other statuses you can use). Using OPEN resets the pointer for READ and WRITE to the first line (so the next READ will begin at the first line, and the next WRITE will write to or overwrite the first line).

In python, you could use `variableforpath=filedialog.askopenfilename()` to request a file to be selected from file explorer (will obtain the file path to this file) and store in variableforpath. Then can do `filetoreadvariable=open(variableforpath,'r')`, if the opened file is just being used for reading (w/ READ), and store in filetoreadvariable.

WRITE in fortran →

`WRITE(*,...)VAR` is essentially `print()` for whatever string is in the ... plus VAR

`WRITE(#,...) VAR` indicates unit number #, so it is writing (again, string in the ... along with whatever VAR holds) into the file using that unit number. Like READ, it will move to the next line of the file to write for every time the function is used (assuming OPEN for the same file is not used in between)

READ in fortran →

`READ(*,...)VAR` asks for user input, much like python's `input()` function, and stores it in VAR

Assuming there is a given unit number (e.g. `READ(#, ...)`), for each READ statement (or iteration of READ), it will read-in the first line (or next line if used again without re-opening the file with OPEN – would reset

pointer for what line to read-in next) of an opened file with a matching unit specifier into the paired variable

e.g.

```
READ(10, '(A)', ERR=740) VAR, VAR2
```

```
READ(10, '(A)', ERR=740) VAR3
```

This would read (from a file corresponding to unit number 10) in the first line into VAR, second into VAR2, and third into VAR3.

For python, you could do `readfilevariable=filetoreadvariable.read().splitlines()` and index `readfilevariable` to read-in specific lines into other variables (e.g. `lineone=readfilevariable[0]` ; will hold first line of file as string).

- `.splitlines()` will make a list of the file's lines (as strings)

- You can then use `.split()` on an indexed line (e.g. `lineone` or `readfilevariable[0]`) to create a string list of all terms (separated by a space) on the indexed line → indexing the result of this will give you a specific term on the line (e.g. `thirdnumber_on_lineone=lineone.split()[2]`) → remember! This indexed term is still string, so if it is a number you want, you also need to convert it to a numerical type (e.g. `int(thirdnumber_on_lineone)`) to use it as such

- If you indexed `lineone` without using `.split()`, you would be indexing a single character

Debugging

- Fortran: Can use VS Community (at least 2019) to open, run, and debug fortran files (do this with Create New Project – on home page > Empty Project (Fortran), then Next to configure new project, then Create > Project > Add Existing Item + add .for file (source file) > click on Solution Explorer on right and click on your .for file – from here it should show you the script and you can then Run w/ green play button and debug); print statements with “print *, __, __, __”

- VB: Can simply open excel with the code – go to macros (in developer) > edit – can debug from here; as alternative to typical printing (since no terminal) while debugging, you can do `Debug.Print(____)` with a variable as input → by hovering over this statement after hitting a breakpoint, you can see the current value of the variable

With source file for fortran, you can run the code. Cannot as easily run fortran code in VS Code.

VS Code (w/ GitHub)

- Refresh: Updates panel on any new changes you or team made → doesn't exactly work at the moment pressing run for the program works to "refresh"
- Sync: pulls from then pushes to github repository
- Stash: like an incomplete commit / saving a rough draft; will remove new, uncommitted edits from workspace (reverts it) and store the edited version in the stash
 - Pop stash will remove draft from stash after applying (replacing the version in the workspace with the draft) it
 - Apply stash will just apply, and retain draft in the stash
- Order of things to do: Refresh → Pull or Sync (mainly to retrieve any of team's changes) → (Optional: stash) → stage changes → WRITE MESSAGE before committing (otherwise will get stuck during the commit; if it gets stuck, would have to close workspace and reopen again – workspace likely, and hopefully, autosaved – and try again but with a message) → Commit → Push or Sync changes (mainly to push your changes)